Derrière les flags

À DIRE

« Les agents ont résolu ces challenges. Nous allons reprendre leurs raisonnements, un à un, avec des schémas et les preuves conservées. On commence par un cas très visuel : un texte transformé en nombres, puis retrouvé grâce aux tables publiques de GPT-2. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l’idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

QUESTIONS PROBABLES

SOURCES

Journaux des sept challenges demandés.

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.

Embeddings — le défi

À DIRE

« On nous donne un fichier appelé Dockerfile.json. Malgré son nom, il contient du JSON et des nombres. Il indique le modèle GPT-2. Il y a 24 groupes de 768 nombres. Le flag a été transformé en cette représentation. Notre mission est de retrouver le texte qui se cache derrière. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le fichier contient session_id, model="gpt2" et inputs_embeds. La forme 24 × 768 est directement vérifiable dans le JSON. Un vecteur est simplement une liste de nombres. Un token est un morceau de texte : parfois un mot, parfois quelques lettres, parfois un signe.

QUESTIONS PROBABLES

Faut-il connaître le machine learning ? Non : il suffit de comprendre une table de correspondance et une addition. Le schéma explique la structure confirmée par la résolution, pas une hypothèse prise pour un fait au début.

SOURCES

LOST_IN_EMBEDDING · JOURNAL 20:05, 20:09 · DOCKERFILE.JSON

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.

SLIDE 3 / 36 · 45–60 secondes

Embeddings — le dictionnaire

À DIRE

« Un embedding, ici, c'est une liste de nombres associée à un morceau de texte. Imaginez un dictionnaire : à gauche, "BA" ; à droite, 768 nombres. GPT-2 possède cette table. Dans notre fichier, une deuxième liste de nombres a été ajoutée pour indiquer la place du morceau dans le texte. Et les deux tables sont disponibles dans le modèle public. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Les tables sont wte, avec 50 257 lignes de tokens, et wpe, avec les positions. Dans ce challenge, vecteur[i] = wte[token_i] + wpe[i]. Les tokens BA, 26 et { correspondent effectivement aux trois premiers tokens retrouvés (4339, 2075, 90). Les barres sont une illustration, pas une projection calculée des 768 dimensions.

QUESTIONS PROBABLES

Un token est-il une lettre ? Pas forcément. Les trois premiers sont BA, 26 et {. Il ne faut donc pas annoncer 24 lettres : il y a 24 morceaux. Les tables ont été téléchargées pendant le CTF après autorisation ; rien n'a été téléchargé pour cette présentation.

SOURCES

LOST_IN_EMBEDDING · CLAUDE/WRITEUP.MD, SECTIONS 2–4

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.

SLIDE 4 / 36 · 45–60 secondes

Embeddings — le chemin inverse

À DIRE

« Prenons le premier groupe. Nous savons qu'il contient les nombres du morceau et ceux de sa position. On retire les nombres de position. Il reste presque exactement une ligne du dictionnaire. On cherche la ligne la plus proche : elle correspond à “BA”. Puis on refait la même chose pour le deuxième groupe, puis le troisième, jusqu'au bout. »

DÉROULÉ DES ANIMATIONS

1. Lire un groupe de nombres.
2. → Retirer sa composante de position.
3. → Chercher le token le plus proche.
4. → Répéter et assembler.

POUR COMPRENDRE

L'agent a chargé les poids publics de GPT-2, soustrait wpe pour les positions 0 à 23, puis comparé chaque vecteur aux 50 257 lignes de wte avec une distance euclidienne. Les indices retrouvés ont été passés au tokenizer pour reconstituer la chaîne.

QUESTIONS PROBABLES

Pourquoi « la plus proche » et pas « identique » ? Les nombres conservés présentent de très petits écarts. Après correction, les distances observées sont de 0,000 à 0,004. Le plus proche voisin suffit. Il n'y a pas de génération de texte ou de prompt envoyé à un chatbot.

SOURCES

LOST_IN_EMBEDDING · CLAUDE/SRC/INVERT.PY · CLAUDE/OUT-01.TXT

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.

SLIDE 5 / 36 · 45–60 secondes

Embeddings — le résultat

À DIRE

« En assemblant les morceaux, on retrouve le flag. La sortie conservée le montre, et les mesures confirment des correspondances presque exactes. La leçon : ici, remplacer le texte par ses embeddings ne le rendait pas secret. On connaissait le modèle, ses tables étaient publiques, et l'opération pouvait être inversée. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le résultat brut, avant soustraction des positions, donnait déjà les bons tokens, avec des distances bien plus grandes. La correction rend la correspondance convaincante : distances de 0,000 à 0,004 et similarités cosinus affichées à 1,000. Le script final est claude/src/invert.py ; la sortie est claude/out-01.txt.

QUESTIONS PROBABLES

Est-ce vrai de tous les embeddings ? Cette démonstration porte sur ces embeddings d'entrée GPT-2, formés par une table de tokens et une table de positions. Il ne faut pas présenter cette recette comme une inversion universelle de tous les vecteurs de tous les modèles.

SOURCES

LOST_IN_EMBEDDING · JOURNAL 20:09 · CLAUDE/OUT-01.TXT

LIMITES ET PROVENANCE

La capture haute résolution a été produite après l'événement à partir de la sortie sauvegardée. Le solve n'a pas été relancé pour cette présentation.

SLIDE 6 / 36 · 45–60 secondes

ID, please — deux portes d’entrée

À DIRE

« Le site propose une connexion avec un mot de passe, ou avec Google. Dans la connexion Google, on transmet au site une attestation qui indique quel compte se connecte. Cette attestation devrait être vérifiée. Or le chemin appelé “legacy” accepte même une attestation sans signature. On peut donc fabriquer ce qu’elle contient. Mais cela ne suffit pas encore : il faut désigner le bon compte. »

DÉROULÉ DES ANIMATIONS

1. Présenter les deux modes de connexion.
2. → Expliquer l’attestation Google en mots simples.
3. → Révéler l’absence de vérification de signature.

POUR COMPRENDRE

Les journaux décrivent POST /member/auth/socialLogin, qui accepte un JWT non signé avec alg:none et répond “legacy compatibility login”. JWT est le format de l’attestation. Le champ sub contient l’identifiant du compte Google. La session créée par l’application est ensuite un cookie opaque, distinct de ce JWT. Les détails du format ne sont pas nécessaires pour suivre la démonstration.

QUESTIONS PROBABLES

Pourquoi une signature ? Elle permet au site de contrôler l’origine et l’intégrité de l’attestation. Lire un numéro dans un document qu’un visiteur peut fabriquer ne prouve pas que ce visiteur possède le compte correspondant.

SOURCES

ID_PLEASE · JOURNAL 18:39, 20:25 · WRITEUP, SECTIONS 1–2

LIMITES ET PROVENANCE

Schéma pédagogique : aucune capture de connexion n’est présentée comme une preuve. L’analyse reprend les observations conservées ; aucune requête n’a été relancée.

SLIDE 7 / 36 · 45–60 secondes

ID, please — la mauvaise identité

À DIRE

« Les premières tentatives étaient intuitives : écrire “je suis admin” dans le document, ou mettre l’email de l’admin. Elles échouaient. Le rôle venait du compte déjà enregistré dans la base, et le site retrouvait ce compte avec un champ appelé sub. Même email, mais autre sub : autre compte, simple utilisateur. Le vrai problème devient donc : comment retrouver le numéro du compte admin ? »

DÉROULÉ DES ANIMATIONS

1. Le champ rôle est ignoré.
2. → Le même email ne relie pas les comptes.
3. → Introduire sub comme le numéro précis du compte.
4. → Reformuler le nouvel objectif.

POUR COMPRENDRE

Le comportement a été vérifié par plusieurs agents : même sub, même membre ; nouveau sub avec un email déjà connu, nouveau membre. Les champs role/isAdmin ne changent pas le rôle. L’enquête a aussi écarté de nombreuses pistes de mutation de comptes et d’injection, mais le public n’a pas besoin d’en mémoriser la liste. L’identifiant sub est la pièce qui relie l’attestation au membre stocké.

QUESTIONS PROBABLES

Pourquoi ne pas inventer un numéro ? Il désignerait généralement un nouveau compte utilisateur. Le résultat recherché est de retrouver la ligne existante de l’admin, pas de créer un compte qui porte un nom convaincant. Les essais de nombres ou mots courants n’ont pas retrouvé le bon compte.

SOURCES

ID_PLEASE · JOURNAL 18:39, 20:12, 20:25 · WRITEUP, SECTION 3

LIMITES ET PROVENANCE

Les résultats présentés sont ceux des essais documentés. Ils n’établissent pas qu’aucune autre attaque ne serait possible. Les identifiants de membres obtenus avant les remises à zéro pouvaient appartenir à d’autres participants.

SLIDE 8 / 36 · 45–60 secondes

ID, please — retrouver le numéro

À DIRE

« Un indice était sur la page d'accueil : le pied de page nommait Tech Corp Chandler et donnait une adresse Gmail. L'agent a utilisé une session Google de test pour chercher cet email via People API. La réponse lui a donné le numéro Google du compte, aussi appelé Gaia ID. Dans ce challenge, c'est le sub attendu. On peut maintenant fabriquer une attestation qui désigne le vrai compte admin. »

DÉROULÉ DES ANIMATIONS

1. Montrer l'indice de la page d'accueil.
2. → Décrire une recherche avec une session Google de test.
3. → Révéler l'identifiant abrégé.
4. → Résumer le changement d'approche.

POUR COMPRENDRE

Le journal de solve et le write-up documentent la recherche de techcorpchandler@gmail.com et le numéro 103465053501192984574. Le script claude/gaia_lookup.py conserve la logique de recherche. Aucun cookie, en-tête d'authentification ou secret de session n'est intégré à la présentation. Le numéro abrégé sur la slide est seulement un repère visuel.

QUESTIONS PROBABLES

Le numéro est-il un mot de passe ? Non : c'est un identifiant. La faille apparaît lorsque le site accepte ce numéro dans une attestation dont il ne vérifie pas l'origine. La présentation n'affirme pas qu'on peut résoudre n'importe quel email Google : elle décrit la recherche qui a fonctionné pour ce compte CTF.

SOURCES

ID_PLEASE · JOURNAL 20:35, 21:25 · WRITEUP, SECTIONS 4–5

LIMITES ET PROVENANCE

Le journal et le write-up documentent le résultat de la recherche. Le script est conservé, mais les cookies sont hors du dépôt. Un code HTTP 401 observé plus tôt ne prouve pas à lui seul qu'un compte existe ou qu'il est social ; le succès final confirme ici le compte ciblé.

SLIDE 9 / 36 · 45–60 secondes

ID, please — une identité usurpée

À DIRE

« Avec le vrai identifiant, l'attestation fabriquée retrouve le membre numéro 1, qui est administrateur. La session obtenue permet alors de lire le flag sur /api/flag. Le journal consigne le solve à 21 h 25. La leçon tient en une phrase : connaître le numéro d'un compte n'est pas une preuve qu'on en est le propriétaire. Le site devait vérifier l'attestation, et pas seulement lire le numéro qu'elle contenait. »

DÉROULÉ DES ANIMATIONS

1. Présenter la chaîne finale.
2. → Révéler les résultats consignés dans le write-up.
3. → Montrer le flag et la leçon.

POUR COMPRENDRE

Le payload final porte le vrai sub et le compte email. Le rôle n'est toujours pas ajouté par le token : il vient de la ligne administrateur déjà stockée. Le write-up transcrit socialLogin avec memberId=1 et role=admin, /api/me avec le membre admin, puis /api/flag avec le flag. Il corrige l'hypothèse précédente sur /api/admin/flag : cette route renvoie 404 même comme admin.

QUESTIONS PROBABLES

Qui a fait quoi ? Le write-up attribue la reconnaissance et les exclusions de pistes à plusieurs agents (pi, Codex, agy, qwen) et le déclic avec le solve final à Claude. Pourquoi tant d'impasses ? L'authentification était faible, mais il fallait d'abord trouver le sub exact ; les changements de rôles ou d'email ne désignaient pas le bon compte.

SOURCES

ID_PLEASE · JOURNAL 21:25 · WRITEUP, SECTIONS 6–8

LIMITES ET PROVENANCE

Preuve disponible : entrée de solve et réponse transcrite dans WRITEUP.md. Aucune capture ni sortie brute finale séparée n'a été trouvée dans les fichiers inspectés. Le panneau de cette slide est une transcription du write-up, pas une capture du service. Aucun exploit n'a été relancé.

Universal translator — retrouver un dictionnaire

À DIRE

« On reçoit un programme destiné à une machine inconnue. Il est composé de petits codes, mais on ne sait plus quelle action chaque code représente. La machine travaille avec une pile : elle dépose des valeurs, les copie, les déplace ou fait des calculs. Heureusement, on nous donne aussi quatre exemples et les résultats qu'ils doivent produire. Ces exemples vont servir à reconstruire le dictionnaire. »

DÉROULÉ DES ANIMATIONS

1. Montrer les codes dont le sens manque.
2. → Expliquer la pile en une phrase.
3. → Montrer les quatre exemples : ils servent de tests.

POUR COMPRENDRE

L'archive contient deux README, quatre échantillons binaires et transmission.bin, un programme de 176 octets. Les codes 0x00 à 0x07 sont une permutation de huit opérations de pile ; les codes 0x10 à 0x19 sont une permutation de dix opérations arithmétiques. Les autres octets ajoutent directement une valeur à la pile. Une pile se lit depuis son sommet, la dernière valeur ajoutée. Les cases affichées sont un schéma, pas un état extrait des échantillons.

QUESTIONS PROBABLES

Pourquoi ne pas essayer les instructions au hasard ? Il y a un ensemble fini de dictionnaires possibles. Les réponses connues permettent de rejeter automatiquement ceux qui ne conviennent pas. Ici, le défi est de retrouver le langage de la machine, puis d'interpréter la transmission.

SOURCES

UNIVERSAL_TRANSLATOR · JOURNAL 18:48–18:51 · README DE L'ARCHIVE

LIMITES ET PROVENANCE

challenge.md est un modèle vide. Le journal conserve le texte fourni par l'utilisateur et les règles lues dans le README.

Universal translator — les tests éliminent les mauvaises traductions

À DIRE

« L'agent essaie les dictionnaires possibles. Pour chaque dictionnaire, il simule les petits programmes d'exemple. Si le résultat est différent de la réponse attendue, le dictionnaire est éliminé. Il procède en deux temps : les opérations de pile, puis les opérations de calcul. À la fin, une seule interprétation satisfait tous les exemples. On peut alors lui confier le message inconnu. »

DÉROULÉ DES ANIMATIONS

1. Attribuer un sens aux codes.
2. → Simuler un exemple.
3. → Comparer, éliminer ; une interprétation reste.
4. → Reprendre la leçon : tous les exemples doivent passer.

POUR COMPRENDRE

Le solve est une recherche exhaustive, pas une déduction au moyen de Z3. stack_candidates.py teste les 8! permutations des opérations de pile contre les exemples 01 et 02 et trouve une seule correspondance. arithmetic_candidates.c teste les 10! permutations arithmétiques. L'exemple 03 laisse 428 candidats non signés ou 346 signés. Avec l'exemple 04, il reste zéro candidat non signé et un seul signé. Le modèle correct utilise des entiers signés sur 64 bits pour MAX, MIN et le décalage à droite.

QUESTIONS PROBABLES

Que signifie « signé » ? La machine doit pouvoir traiter certaines valeurs comme négatives. Une mauvaise interprétation de ces valeurs change les comparaisons et les décalages. Ce détail est essentiel au solve, mais n'est pas nécessaire pour comprendre le principe des tests successifs.

SOURCES

UNIVERSAL_TRANSLATOR · JOURNAL 18:51–18:53 · CODEX/SOLVER-OUTPUT.TXT

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.

Universal translator — la transmission lue

À DIRE

« Le dictionnaire est maintenant retrouvé. Par exemple, 03 signifie copier le sommet de la pile, 04 signifie émettre une valeur, et 19 signifie additionner. L'agent utilise cette machine reconstruite pour lire transmission.bin. Elle produit des octets qui, convertis en texte, donnent le flag. La preuve est la sortie conservée du solveur. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le mapping complet de pile est 00=ROT, 01=TUCK, 02=OVER, 03=DUP, 04=EMIT, 05=NIP, 06=SWAP, 07=DROP. Le mapping arithmétique est 10=RSHIFT, 11=XOR, 12=LSHIFT, 13=*, 14=AND, 15=MIN, 16=MAX, 17=-, 18=OR, 19=+. Les codes sont en hexadécimal. La transmission produit les octets 42 41 32 36 7b… qui deviennent BA26{0n3_st4ck_f0rth_4ll} en ASCII. Les scripts finaux sont codex/stack_candidates.py, codex/generate_data.py et codex/arithmetic_candidates.c.

QUESTIONS PROBABLES

L'agent a-t-il contacté un serveur ? Aucun serveur cible n'est spécifié dans les archives de ce challenge. Le travail documenté est local, sur les fichiers fournis. La présentation ne relance pas le solveur.

SOURCES

UNIVERSAL_TRANSLATOR · CODEX/SOLVER-OUTPUT.TXT · JOURNAL 18:53

LIMITES ET PROVENANCE

La capture a été annotée après l'événement. La correspondance et les octets décodés sont aussi consignés dans la sortie texte conservée.

Silent beacon — le programme et le leurre

À DIRE

« On reçoit une copie de la mémoire d'une balise. Son programme demande un code d'activation de huit caractères hexadécimaux. Si le code est correct, il décode un enregistrement. En cherchant les textes lisibles dans le fichier, on tombe aussi sur un flag tout prêt. C'est un leurre. L'objectif est de comprendre le contrôle du code, puis de suivre le vrai décodage. »

DÉROULÉ DES ANIMATIONS

1. Présenter le dump de mémoire.
2. → Présenter le contrôle du code.
3. → Montrer le leurre en clair.
4. → Dire ce qui fera réellement preuve.

POUR COMPRENDRE

Le dump beacon_flash.bin est identifié comme un firmware ESP32-C6. Le contrôle d'activation et le décodage sont dans le programme fourni. Le faux flag BA26{n0t_th15_0n3_k33p_d1gg1ng} est visible en clair à l'offset 0x19d00. La reconstruction montre que le programme en lit seulement le premier caractère pour un message de démarrage ; le chemin qui affiche UNSEALED utilise une autre zone de données.

QUESTIONS PROBABLES

Le faux flag a-t-il été soumis ? Les archives ne le disent pas. Ne pas le prétendre. Avions-nous l'appareil physique ? Les sources documentent un dump et un solveur hors ligne, pas une activation sur un appareil.

SOURCES

SILENT_BEACON · JOURNAL 18:45, 18:49 · BACKFILL 20:57

LIMITES ET PROVENANCE

Le texte complet du challenge, les commandes exactes de désassemblage et une éventuelle activation réelle ne sont pas conservés. Le schéma mémoire est pédagogique, pas une carte à l'échelle.

Silent beacon — retrouver le code attendu

À DIRE

« En lisant le programme, l'agent repère deux textes d'identité. Le programme transforme ces deux textes en nombres, puis les combine pour calculer le code attendu. L'agent reproduit exactement ces opérations et retrouve B80FEF82. Il peut ensuite reproduire le décodage des 31 octets qui contiennent le vrai enregistrement. »

DÉROULÉ DES ANIMATIONS

1. Les deux textes sont connus : ils sont dans le dump.
2. → Le programme décrit leur transformation.
3. → Le code attendu est retrouvé.
4. → Le même travail permet de reproduire le décodage.

POUR COMPRENDRE

Le contrôle applique FNV-1a 32 bits à LilyGO T7-C6 et à BA26-BEACON-01. Le code attendu est une rotation de 7 bits vers la gauche du premier hash, combinée par XOR avec le second. Les hashes conservés sont 56ea3ff6 pour LilyGO T7-C6 et cd1014a9 pour BA26-BEACON-01. Leur ordre compte : une tentative initiale les a inversés, donnant DEE06B10 et du texte incohérent. Le décodeur utilise les deux hashes comme état 64 bits d'un xorshift, avec des décalages de 13, 7 et 17 bits, puis un XOR avec chaque octet chiffré.

QUESTIONS PROBABLES

Que veut dire XOR ? C'est une opération sur les bits. Pour l'exposé, l'idée utile est que l'agent a recopié la suite d'opérations du firmware. L'équation n'est pas nécessaire pour suivre le raisonnement. Le code retrouvé ouvre logiquement la branche de succès ; les archives ne prouvent pas une saisie sur un service réel.

SOURCES

SILENT_BEACON · CODEX/SOLVE.PY · JOURNAL 18:49

LIMITES ET PROVENANCE

L'erreur d'ordre des deux hashes est documentée dans le journal, mais sa sortie intermédiaire n'a pas été sauvegardée.

Silent beacon — la preuve du décodage

À DIRE

« Le solveur retrouve le code attendu et décode l'enregistrement. Cette fois, le flag vient bien du chemin de succès du programme, et il est enregistré dans la sortie du solveur. On sait donc expliquer comment il a été obtenu. C'est la différence avec le faux flag trouvé en clair : l'analyse relie le résultat au mécanisme réel du challenge. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

codex/solve.py lit le dump, récupère les textes d'identité, calcule les hashes et le code, puis refait le xorshift et les XOR sur les 31 octets. La sortie conservée indique activation code=B80FEF82 et record=BA26{x0rsh1ft_s34ls_th3_b34c0n}. Les étapes techniques détaillées sont dans le journal original de Codex et dans la reconstruction à partir du désassemblage conservé.

QUESTIONS PROBABLES

Est-ce une preuve de déverrouillage matériel ? Non : la preuve documentée est le calcul et le décodage hors ligne. Le journal ne conserve aucune trace d'une saisie du code sur un appareil ou un service. Ce que l'on peut affirmer est que le solveur reproduit la logique analysée et retrouve le résultat journalisé.

SOURCES

SILENT_BEACON · CLAUDE/OUT-03-SOLVE.TXT · JOURNAL 18:49

LIMITES ET PROVENANCE

La capture de sortie a été produite lors du backfill à partir d'une reproduction hors ligne. Elle concorde avec le code d'activation et le flag consignés par Codex pendant le solve.

SLIDE 16 / 36 · 45–60 secondes

Wrong path — sortir du dossier

À DIRE

« Cette application permet de déposer puis de lire des fichiers. Elle est censée nous limiter à notre dossier. Mais un de ses points d'entrée accepte un chemin sans le contrôler. En ajoutant “point point slash”, on remonte dans les dossiers. L'agent a ainsi pu lire le code de l'application puis sa base de comptes. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le journal décrit une application Flask appelée FileVault. Les routes /view?filename= et /raw?filename= ouvrent un chemin construit avec os.path.join sans assainissement ; /download utilise send_from_directory et bloque la traversée. Le point de départ est /app/uploads/<utilisateur>/. La lecture ../../app.py confirme l'emplacement de la base /app/challenge.db. Le test initial contre /etc/passwd est également documenté.

QUESTIONS PROBABLES

Pourquoi le chemin ../ est-il spécial ? Il signifie « le dossier parent ». Si une application le laisse agir dans une entrée utilisateur, le chemin peut quitter le répertoire prévu. Il ne faut pas dire que toutes les routes étaient vulnérables : /download était précisément le chemin sûr.

SOURCES

WRONG_PATH · CLAUDE · JOURNAL, APP ET EXPLOITATION

LIMITES ET PROVENANCE

Aucune capture de l'interface web ni requête/réponse brute n'a été conservée. Ce schéma reconstitue le mécanisme décrit dans le journal ; les horaires intermédiaires sont approximatifs.

Wrong path — le faux flag

À DIRE

« La base contient un mot de passe qui ressemble à un flag. L'agent pense d'abord avoir fini, puis corrige cette conclusion : c'était un piège. Les vrais identifiants administrateur sont dans la même base. Une fois connecté comme admin, il voit un fichier qui indique que le vrai flag a été caché dans un labyrinthe. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le journal contient une correction explicite : la chaîne de type BA26-dftwp-… est un leurre, pas le résultat final. La lecture de /proc/self/environ a révélé SECRET_KEY et confirmé la localisation des identifiants. Le compte admin de challenge.db a permis l'accès à /dashboard, où figuraient flag.txt et flag.maze. Le solve final utilise les identifiants ; il ne faut pas inventer une falsification de session avec SECRET_KEY.

QUESTIONS PROBABLES

Comment sait-on que le premier flag était faux ? Le journal le corrige explicitement et le résultat réel est conservé dans claude/out-01.txt. La présentation suit cette correction, pas le premier résumé obsolète.

SOURCES

WRONG_PATH · CLAUDE · JOURNAL, CORRECTION ET RECONSTRUCTION

LIMITES ET PROVENANCE

Le journal reconstitue l'ordre général des étapes après l'événement. Il n'existe pas de traces HTTP permettant de dater chaque action précisément.

SLIDE 18 / 36 · 45–60 secondes

Wrong path — le labyrinthe

À DIRE

« Le fichier est un labyrinthe en texte : des murs, des couloirs et des lettres. Le solveur explore les passages depuis l'entrée jusqu'à la sortie. Il conserve ce chemin, puis lit les caractères qui s'y trouvent. Ensemble, ils forment le vrai flag. Le dessin vert est un rendu du chemin retrouvé à partir du fichier conservé. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le labyrinthe mesure 201 × 201. Les deux portes sont en (ligne 1, colonne 0) et (ligne 199, colonne 200). claude/solve.py utilise une recherche en largeur, ou BFS, puis remonte les prédécesseurs pour reconstruire le chemin. La sortie sauvegardée indique une longueur de 2 783 positions. Le flag complet est BA26{tHis_Is_a_rEally_loNG_flag_ThAt_is_Very_DiFFicult_to_gUess_bUT_you_cAn_find_It_in_tHE_mAze}.

QUESTIONS PROBABLES

Qu'est-ce qu'une recherche en largeur ? Elle visite d'abord les cases voisines, puis leurs voisines, couche par couche. Le public peut retenir « explorer les couloirs en conservant d'où l'on vient ». Le chemin vert représente le résultat conservé, pas une animation de l'exécution d'origine.

SOURCES

WRONG_PATH · CLAUDE/SOLVE.PY · OUT-01.TXT · RENDU DU LABYRINTHE

LIMITES ET PROVENANCE

Le dessin a été créé après l'événement à partir du labyrinthe local. La chaîne sur la slide est volontairement abrégée ; la preuve et les notes donnent le résultat complet.

SLIDE 19 / 36 · 45–60 secondes

Peak privilege — le défi

À DIRE

« Ce système décide des droits à partir d'une image de badge. Il classe chaque image comme employé, invalide ou administrateur. Pour obtenir le flag, il faut que son score ADMIN atteigne 99,5 %. On ne possède pas le modèle, mais le service donne ses scores. C'est cette information qui va guider les modifications de l'image. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

L'entrée est une image RGB 256 × 176. Le traitement connu découpe un carré 96 × 96, le passe en niveaux de gris puis moyenne des blocs 3 × 3 pour obtenir une grille 32 × 32. L'agent peut donc contrôler 1 024 valeurs via des blocs de pixels. Les poids du modèle ne sont pas fournis. Le badge employé initial a un score EMPLOYEE proche de 1 et ADMIN d'environ $2{,}3 \times 10^{-23}$.

QUESTIONS PROBABLES

Est-ce une reconnaissance de carte d'identité ? Les archives décrivent un scanner artificiel NeuralBadge de CTF avec trois classes. Ne pas extrapoler ce résultat à un produit réel. Le score renvoyé est une sortie du modèle, pas une garantie de probabilité réelle.

SOURCES

PEAK_PRIVILEGE · CLAUDE · JOURNAL SETUP · PREPROCESS.PY

LIMITES ET PROVENANCE

La description vient du journal, car challenge.md est un modèle vide. L'image employé est un fichier fourni ; le scanner dessiné est une illustration.

SLIDE 20 / 36 · 45–60 secondes

Peak privilege — plus chaud, plus froid

À DIRE

« L'idée ressemble à un jeu de "plus chaud, plus froid". L'agent fabrique plusieurs petites variations du badge. Il compare leurs scores et estime dans quelle direction changer l'image pour être davantage classé ADMIN. Puis il recommence. Il ne choisit pas simplement une image au hasard : il utilise les différences entre les scores pour orienter la recherche. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

La méthode est une optimisation NES en boîte noire. Chaque lot contient 20 paires de perturbations gaussiennes opposées, soit 40 images. Le solveur estime la direction d'amélioration de log(pADMIN) − log(max(pEMPLOYEE,pINVALID)), normalise cette direction puis met à jour la grille. Il part du badge employé ; les essais constants et aléatoires avaient surtout donné INVALID. Deux phases sont documentées : sigma=0,06, pas=0,08 puis 0,28.

QUESTIONS PROBABLES

Pourquoi les scores aident-ils autant ? Un simple « non » ne dit pas quelle modification est meilleure. Des scores permettent de comparer les variantes. « Plus chaud, plus froid » est une analogie : la méthode réelle utilise des différences de scores et une estimation de gradient, pas une série de victoires à chaque essai.

SOURCES

PEAK_PRIVILEGE · CLAUDE/WORK/NES.PY · JOURNAL SOLVED

LIMITES ET PROVENANCE

Les variantes et flèches sont schématiques. Aucune série complète de scores par requête n'a été conservée, donc aucun graphique de progression n'est inventé.

SLIDE 21 / 36 · 45–60 secondes

Peak privilege — le badge accepté

À DIRE

« Le badge final est visuellement étrange. Pourtant, le journal rapporte un score ADMIN de 99,6 %, au-dessus du seuil de 99,5 %, puis l'accès accordé et le flag. L'image gagnante a été conservée. La leçon de ce challenge est que le modèle donne à l'attaquant une mesure qui l'aide à fabriquer une entrée acceptée. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le journal rapporte environ 113 appels par lots de 40 images pour l'ensemble de la résolution ; ces nombres restent approximatifs. La seconde phase atteint le seuil après environ 53 itérations. Le badge est claude/work/admin_badge.png, avec la grille gagnante best_grid.npy. Le solveur final est nes.py. Le résultat et le flag sont consignés dans la section SOLVED.

QUESTIONS PROBABLES

La capture prouve-t-elle à elle seule le score ? Non : elle montre le badge gagnant. Le score, l'accès accordé et le flag viennent du journal. Aucun log brut des réponses du service n'est gardé ; la présentation le dit explicitement dans la provenance.

SOURCES

PEAK_PRIVILEGE · CLAUDE · JOURNAL SOLVED · ADMIN_BADGE.PNG

LIMITES ET PROVENANCE

Les notes originales sont non datées ; les horaires de la reconstruction viennent des dates de fichiers. Le score de 0,996 est rapporté dans le journal, sans réponse HTTP brute conservée.

SLIDE 22 / 36 · 45–60 secondes

Rucher — les deux fichiers

À DIRE

« On nous donne dix secondes d'enregistrement radio et une copie du programme d'une balance connectée. La balance communique avec une passerelle. Le secret est dans les messages radio ; le programme nous aide à comprendre comment ces messages sont construits et protégés. Aucun des deux fichiers ne suffit à lui seul pour raconter toute la résolution. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

La capture est un fichier SigMF de 80 Mo, échantillonné à 4 millions d'échantillons par seconde et centré sur 2 475 MHz. Le firmware est un dump ESP32-C6 partiel. Ses chaînes et sa structure indiquent Zigbee. La fréquence correspond au canal 25 d'IEEE 802.15.4. La zone NVS conservée est vierge : pas de clé réseau déjà provisionnée à récupérer dans ce dump.

QUESTIONS PROBABLES

Qu'est-ce qu'un firmware ? Le programme embarqué dans l'appareil. Qu'est-ce qu'un enregistrement radio ? Ici, une suite d'échantillons du signal, pas encore une liste de messages. Il faut d'abord les décoder.

SOURCES

RUCHER · CLAUDE · JOURNAL SECTIONS 0–1 · ARTEFACTS FOURNIS

LIMITES ET PROVENANCE

La chronologie du journal rucher a été reconstruite après l'événement à partir des fichiers. Le dump est tronqué ; la présentation ne suppose pas l'existence de partitions absentes.

SLIDE 23 / 36 · 45–60 secondes

Rucher — des ondes aux paquets

À DIRE

« Au début, on n'a que des ondes enregistrées. L'agent reconnaît le protocole Zigbee et écrit un décodeur. Il compare le signal aux motifs attendus pour retrouver les nombres envoyés. Les paquets possèdent un contrôle d'erreur : c'est un bon moyen de vérifier le décodage. Résultat : les 42 paquets passent ce contrôle. On peut maintenant analyser les messages. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Les scripts decode154.py et extract_all.py détectent les salves et emploient un filtre adapté complexe aux motifs O-QPSK/DSSS. Le suréchantillonnage ×4 améliore la précision. Le FCS CRC-16 sert à confirmer les conventions de modulation et les octets. Le résultat est enregistré dans claude/rucher.pcap, avec 42 salves décodées et des FCS valides. La capture comprend l'association puis des rapports périodiques.

QUESTIONS PROBABLES

Le contrôle d'erreur déchiffre-t-il le message ? Non. Il confirme que les bits radio ont été lus correctement. Le contenu peut encore être chiffré. C'est pour cela que le décodage et le déchiffrement sont deux étapes distinctes.

SOURCES

RUCHER · CLAUDE/DECODE154.PY · EXTRACT_ALL.PY · JOURNAL SECTION 2

LIMITES ET PROVENANCE

L'onde dessinée est une illustration du signal, pas une courbe mesurée. La quantité 42/42 vient du journal et du fichier de paquets conservé.

SLIDE 24 / 36 · 45–60 secondes

Rucher — la clé réseau

À DIRE

« La capture commence assez tôt pour contenir l'arrivée de la balance sur le réseau. Pendant cet échange, la passerelle lui transmet une clé réseau. Cette transmission est protégée à partir d'une clé par défaut connue. L'agent peut donc retrouver la clé réseau et ouvrir les rapports suivants. Ces rapports contiennent six fragments, mais ils ne donnent pas encore le flag en clair. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le message APS Transport-Key est protégé avec une clé de transport dérivée de ZigBeeAlliance09 par le hachage Zigbee AES-MMO avec l'octet 0x00. Il ne faut pas dire que la clé par défaut est directement appliquée au paquet sans cette dérivation. AES-CCM* permet de déchiffrer et de vérifier le MIC. La clé réseau récupérée est b8d8e46f82f99446568d2d30e9512c16. Les rapports NWK vérifiés contiennent six fragments indexés de 0 à 5, totalisant 46 octets.

QUESTIONS PROBABLES

La clé est-elle envoyée en clair ? Non : elle est chiffrée, mais sa protection dérive d'une clé par défaut connue. Pourquoi faut-il capturer l'appairage ? Dans ce solve, c'est l'échange conservé qui fournit la clé réseau nécessaire pour lire les rapports.

SOURCES

RUCHER · CLAUDE/ZBCRYPTO.PY · JOURNAL SECTION 3

LIMITES ET PROVENANCE

Les mécanismes de dérivation et de vérification sont simplifiés sur la slide et détaillés ici. Aucun trafic radio n'a été capturé ou rejoué pour cette présentation.

SLIDE 25 / 36 · 45–60 secondes

Rucher — la couche liée à l’appareil

À DIRE

« Le firmware révèle une dernière couche de chiffrement. La clé de chaque morceau combine trois choses : l’identité de la balance, le mot “propolis” et le numéro du fragment. L’identité est visible dans les paquets ; la recette est dans le programme. En réunissant les deux, l’agent déchiffre les six fragments et retrouve le flag. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l’idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

L’algorithme reconnu est RC4, réinitialisé pour chaque fragment. La clé exacte est EUI-64 en ordre little-endian, suivie des octets de “propolis”, puis d’un octet d’index. L’EUI-64 de la balance est 40:4c:ca:ff:fe:46:87:c4, soit c48746feffca4c40 en little-endian. Le solveur final claude/solve_final.py prend le pcap, récupère la clé réseau, déchiffre les rapports et la couche RC4, puis concatène les fragments. La sortie conservée confirme le flag affiché.

QUESTIONS PROBABLES

L’identité de la balance est-elle un secret ? Elle est observable dans la capture. Ici, elle personnalise le chiffrement, mais elle ne suffit pas à garder le contenu secret une fois le mécanisme compris. Pourquoi le firmware seul ne donne-t-il pas le flag ? Il manque les messages de cette balance et son identité observée.

SOURCES

RUCHER · CLAUDE/SOLVE_FINAL.PY · OUT-01-SOLVE.TXT · JOURNAL SECTION 4

LIMITES ET PROVENANCE

Le résultat montré provient d’une reproduction hors ligne conservée lors du backfill. Cette présentation n’a relancé aucun solveur. Les premiers journaux d’outils et l’essai initial de démodulation ne sont pas conservés.

SLIDE 26 / 36 · 45–60 secondes

La mission : 22 signatures

À DIRE

« Le programme simule une radio de Formule 1. Pour obtenir le flag, il faut faire accepter exactement “Box Box Box!” aux 22 pilotes. Mais envoyer ces mots ne suffit pas : chaque pilote demande une signature valide. Une signature est une preuve mathématique liée au message. La clé privée permet de signer ; la clé publique permet de vérifier. Ici, nous connaissons les clés publiques, car elles sont dans le binaire. Il nous manque donc la capacité de signer. »

DÉROULÉ DES ANIMATIONS

1. Écran initial : présenter le message cible.
2. → Signature : expliquer clé privée et clé publique.
3. → Vérification : insister sur les 22 clés distinctes.

POUR COMPRENDRE

Le binaire fourni est un ELF x86-64 avec symboles et informations de debug. Les fonctions pertinentes sont menu_spy, menu_send, ecdsa_sign et ecdsa_verify. menu_send vérifie la signature ECDSA P-256 et marque le pilote si le texte est exactement “Box Box Box!”. Le programme affiche le flag lorsque les 22 indicateurs sont activés. Les tableaux PILOT_QX et PILOT_QY contiennent les coordonnées des 22 clés publiques.

QUESTIONS PROBABLES

Pourquoi lire une clé publique n’est-il pas déjà une faille ? Elle est faite pour être connue. On doit pouvoir vérifier une signature sans pouvoir retrouver la clé privée. C’est une autre faiblesse qui permet ici de franchir cette séparation.

SOURCES

driver_radio/JOURNAL.md : entrées 20:00 et 20:05 ; claude/send_exploit.py.

LIMITES ET PROVENANCE

challenge.md est aussi un modèle vide dans driver_radio. Le contexte de la slide vient du journal et de la logique documentée du binaire. Le dessin de radio est une illustration pédagogique, pas une capture du programme.

SLIDE 27 / 36 · 60–75 secondes

La faille : 255 nonces possibles

À DIRE

« Pour signer, le programme utilise un nombre secret temporaire. On l'appelle le nonce. Ce nombre devrait être difficile à deviner. Ici, il est forcément entre 1 et 255. Le menu Spy nous montre une signature : elle contient un nombre calculé à partir de ce nonce. L'agent essaie les 255 possibilités et compare. Quand il retrouve la même valeur, il a retrouvé le nonce. Mais il reste une étape : retrouver le message signé. »

DÉROULÉ DES ANIMATIONS

1. Écran initial : montrer que les 255 cases représentent toute la recherche.
2. → Balayage : expliquer « calculer, comparer ». Le balayage est illustratif, sans valeur gagnante inventée.
3. → Spy : distinguer le texte chiffré et la signature visible. Ouvrir la preuve si utile.

POUR COMPRENDRE

Le journal donne k = (rand() % 255) + 1 et r = (k·G).x mod n. G est le point de base de la courbe ; n est son ordre. Le solveur prépare un dictionnaire des valeurs de r associées aux 255 k possibles. Ce calcul est fait une fois, puis r sert de clé de recherche. Le message est montré chiffré en AES-CTR, mais la signature (r,s) reste visible. La faille exposée concerne la génération du nonce ECDSA.

QUESTIONS PROBABLES

Le nonce est-il la clé privée ? Non. C'est un nombre utilisé pour une signature. Le retrouver aide à calculer la clé privée quand on connaît aussi le message et la signature. Le détail mathématique est dans les notes, et la preuve du bug est ouvrable sur la slide.

SOURCES

driver_radio/JOURNAL.md : 20:05 et 20:15 ; claude/recover.py, dictionnaire rk ; screenshots/claude-02-nonce-vuln.annotated.png.

LIMITES ET PROVENANCE

Le fichier de décompilation conservé est vide. La région vulnérable est documentée dans le journal et la capture annotée ; le désassemblage complet n'a pas été gardé. Cette capture est une preuve documentaire, pas du code extrait à nouveau pour cette présentation.

SLIDE 28 / 36 · 60–75 secondes

Le déclic : tester les messages possibles

À DIRE

« La première idée était de reconstruire le générateur aléatoire à partir de l'heure de démarrage, pour reproduire les messages. L'agent a ensuite trouvé une approche plus directe. Les messages radio viennent de quelques modèles : un mode moteur, un plan, une fenêtre d'arrêt… Les variables ont de petites plages de valeurs. Au lieu de déchiffrer le texte, il peut donc lister les textes possibles. Pour chaque texte, il calcule son empreinte SHA-256 et en déduit une clé candidate. La clé publique va permettre de savoir quelle candidate est correcte. »

DÉROULÉ DES ANIMATIONS

1. Écran initial : résumer la première piste, sans s'y attarder.
2. → Modèles de messages : lire un seul exemple à voix haute. Les mots entre accolades sont des variables explicatives.
3. → Environ 800 : expliquer pourquoi une énumération devient réaliste.

POUR COMPRENDRE

Le journal décrit environ 800 messages par pilote, issus de huit types de messages et de petites plages de valeurs, volontairement élargies. recover.py conserve des candidats supplémentaires pour couvrir ces plages. Le fichier permet de vérifier les formats affichés. Le nom du pilote fait partie du message : "[pilote]" sur la slide est une abréviation pédagogique. Les chaînes exactes incluent le nom complet.

QUESTIONS PROBABLES

L'agent a-t-il cassé AES ? Le solve final n'en a pas besoin : il énumère les textes plausibles et vérifie la clé privée résultante contre la clé publique connue. Pourquoi abandonner l'horloge ? La nouvelle approche élimine la dépendance à la seed, à l'heure et à l'alignement des appels de rand().

SOURCES

driver_radio/JOURNAL.md : 20:07, 20:12, 20:15 ; claude/recover.py, fonction enum().

LIMITES ET PROVENANCE

La validation locale envisagée pour la première piste a été bloquée par l'absence de libcrypto.so.3 pour x86-64. Les exemples montrés sont des modèles tirés du solveur, pas une transcription d'un échange précis avec un pilote.

SLIDE 29 / 36 · 75–90 secondes

La clé candidate et le test de confirmation

À DIRE

« Pour chaque message possible, on combine trois informations : la signature observée, le petit nombre secret retrouvé, et le message qu'on essaie. Cela donne une clé privée candidate. Il faut ensuite vérifier qu'elle appartient bien au pilote. Sa clé publique permet ce contrôle. L'agent ne s'arrête donc pas à une supposition : il confirme les clés une par une. Les 22 ont été retrouvées. »

DÉROULÉ DES ANIMATIONS

1. Présenter les trois informations connues.
2. → Elles permettent de calculer une clé candidate.
3. → La clé publique confirme ou rejette la candidate.
4. → Les 22 pilotes ont une clé confirmée.

POUR COMPRENDRE

Équation ECDSA documentée : $s = k^{-1}(z + r \cdot d) \bmod n$. Multiplier par k, soustraire z, puis multiplier par l'inverse de r donne $d = r^{-1}(s \cdot k - z) \bmod n$. $z = \text{SHA256}(\text{message})$. r^{-1} désigne l'inverse multiplicatif modulo n, pas un inverse décimal. d est un entier privé, G le point de base, Q la clé publique. Le test $d \cdot G = Q$ vérifie la candidate. Dans recover.py, il correspond à eqpt(mulG(d), Q).

QUESTIONS PROBABLES

Pourquoi faut-il deviner le message ? Sans z, le calcul de d n'est pas déterminé. Pourquoi plusieurs signatures ont-elles été récupérées ? Le journal indique dix observations par pilote, soit 220 signatures ; le solveur parcourt les signatures jusqu'à trouver une clé confirmée. Il ne faut donc pas dire que l'opération historique n'a demandé qu'une seule observation par pilote.

SOURCES

driver_radio/JOURNAL.md : 20:15–20:16 ; claude/recover.py, lignes de récupération et comparaison ; claude/crypto.py.

LIMITES ET PROVENANCE

Cette présentation lit les scripts et les résultats conservés. Elle ne recalcule ni les clés ni les signatures. Les clés privées n'ont pas besoin d'être affichées pour expliquer ou démontrer le résultat.

SLIDE 30 / 36 · 45–60 secondes

Le résultat : 22 commandes acceptées

À DIRE

« Avec les 22 clés privées, l'agent peut fabriquer une signature valide du message "Box Box Box!" pour chaque pilote. Il les vérifie localement, puis les envoie lors d'une nouvelle connexion. Le journal et la réponse enregistrée montrent les 22 commandes acceptées, puis le flag. La leçon est précise : dans ce challenge, le petit nonce et le faible nombre de messages possibles ont permis de retrouver les clés. La sécurité de la signature dépendait aussi de la qualité de ces entrées. »

DÉROULÉ DES ANIMATIONS

1. Écran initial : laisser les 22 cases apparaître. C'est une illustration du résultat enregistré.
2. → Preuve : montrer le 22/22 et la ligne FLAG, agrandir si nécessaire.
3. → Leçon : reprendre les deux faiblesses ensemble.

POUR COMPRENDRE

Chaîne de fichiers historique : harvest.py → sigs.json → recover.py → priv.json → send_exploit.py → final_resp.txt. Les clés sont statiques entre connexions selon le journal ; l'agent a ainsi pu récupérer hors ligne puis signer dans une nouvelle connexion. Le flag enregistré est BA26{T0t0_1ts_C4ll3d_4_M0t0r_R4c3_N0t_S3cur3_Crypt0}.

QUESTIONS PROBABLES

Est-ce une cassure générale d'ECDSA ? Les archives montrent une exploitation de cette implémentation : nonce de 1 à 255 et messages énumérables. Ne pas généraliser le résultat à toutes les implémentations ou à tous les usages de signatures.

SOURCES

driver_radio/JOURNAL.md : 20:19–20:20 ; claude/final_resp.txt ; claude/send_exploit.py ; screenshots/claude-01-flag.annotated.png.

LIMITES ET PROVENANCE

L'annotation de la capture a été ajoutée après l'événement. Le résultat est corroboré par final_resp.txt et le journal de solve. Les timings d'animation n'indiquent pas la vitesse réelle d'exploitation.

SLIDE 31 / 36 · 45–60 secondes

Tapo — le défi

À DIRE

« Une caméra, c’est aussi un petit ordinateur. Dans ce challenge, on récupère une copie de sa mémoire et on cherche un flag. Première idée : chercher directement le texte. Rien. La caméra avait donc bien rangé son secret… mais elle avait gardé tous les tiroirs. Il faut comprendre comment les fichiers sont stockés. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l’idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le PowerPoint fourni décrit une Tapo C500 2.0 et un dump de 8 Mio. Il rapporte une première extraction incomplète et des recherches textuelles infructueuses. Ces faits viennent du support, le dump lui-même n’ayant pas été fourni dans cette demande.

QUESTIONS PROBABLES

Un dump est-il une vidéo ? Non : ici, c’est une copie brute de la mémoire flash, qui contient le logiciel et les données de l’appareil.

SOURCES

TAPO · POWERPOINT FOURNI, SLIDES 2–5

LIMITES ET PROVENANCE

Présentation reconstruite depuis le PowerPoint et les fichiers fournis. Aucun appareil contacté, aucun script exécuté.

Tapo — la zone oubliée

À DIRE

« Le tournant, c'est une zone que la première extraction avait manquée. Imaginez un livre sans couverture : impossible de lire son titre, mais la table des matières et les pages existent toujours. Ici, les métadonnées indiquent encore où retrouver les fichiers. On peut donc les récupérer directement. Le livre était mal étiqueté, pas vide. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le script utilise une base à 0x1c0000, décompresse deux blocs de métadonnées XZ aux offsets 0x3a8236 et 0x3a887c, puis une table de fragments à 0x3aa18a. La fonction fragment_file lit les informations d'inode et de fragment pour extraire les données. Précision de la version (2), slide 14 : read_bytes lit le dump ; xz décompresse les blocs ; les tables décrivent les fichiers et la position de leurs données. fragment_file repère le bloc, le décompresse si nécessaire, puis découpe les octets du fichier. Les positions sont déjà connues et codées dans le script : il est spécifique à ce dump, sans découverte automatique. unlock contient le programme extrait, encrypted contient le message chiffré. Le PPT rapporte un superbloc SquashFS non reconnaissable.

QUESTIONS PROBABLES

L'en-tête a-t-il été saboté volontairement ? Le support ne le démontre pas. On peut seulement dire qu'il n'était pas reconnaissable et que des métadonnées restaient exploitables.

SOURCES

TAPO · POWERPOINT SLIDE 6 · SOLVE.PY

LIMITES ET PROVENANCE

L'analogie du livre est pédagogique. Les offsets sont lisibles dans le script, mais l'extraction n'a pas été relancée.

SLIDE 33 / 36 · 45–60 secondes

Tapo — les ingrédients

À DIRE

« On trouve le message chiffré, un petit programme nommé ctf_unlock, et le script de démarrage qui dit comment les relier. Ce dernier utilise l'identifiant de la caméra. Autrement dit, la caméra a fermé le coffre, mais elle a laissé la recette d'ouverture dans le même carton. Il reste à comprendre cette recette. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Le fichier ctf_unlock fourni est un exécutable ELF 32 bits MIPS, statique et stripped. Le PPT indique que le script de démarrage lit devId, le transmet au programme avec flag.enc et écrit le résultat dans /tmp/flag.txt. solve.py récupère devId dans le dump via une expression régulière.

QUESTIONS PROBABLES

L'identifiant est-il une clé secrète inaccessible ? Dans ce challenge, il est récupérable dans le dump. Cette observation ne démontre pas une vulnérabilité générale de tous les produits Tapo.

SOURCES

TAPO · POWERPOINT SLIDES 7–8 · CTF_UNLOCK · FLAG.ENC · SOLVE.PY

LIMITES ET PROVENANCE

Le script de démarrage et le désassemblage sont décrits dans le PowerPoint ; ils ne sont pas fournis séparément. Le binaire n'a pas été exécuté.

Tapo — le masque

À DIRE

« Le programme fabrique une suite de nombres à partir de l'identifiant, d'un petit texte fixe et d'un compteur. Cette suite sert de masque. Le chiffrement utilise une opération appelée XOR : avec le même masque, on peut retrouver le message d'origine. L'agent a donc traduit la recette en Python. Pas de boule de cristal : les ingrédients étaient déjà dans le fichier. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Recette lisible dans solve.py : SHA256(devId_ASCII || C500-CTF-v1 || LE32(compteur)), compteur à partir de zéro. Chaque empreinte donne 32 octets ; deux empreintes couvrent les 35 octets chiffrés. Le XOR est appliqué octet par octet. Le devId est utilisé tel quel en ASCII, sans conversion hexadécimale. Précisions de la version (2), slide 15 : re.search récupère la valeur du champ devId ; struct.pack encode le compteur sur quatre octets little-endian ; digest produit 32 octets par empreinte ; join assemble les blocs. Le calcul du nombre de blocs arrondit vers le haut : 35 octets nécessitent les compteurs 0 et 1. zip associe les octets chiffrés au flux et s'arrête au plus court : seuls 35 octets servent. Le symbole ^ réalise XOR, bytes reconstitue le résultat et decode le transforme en texte. Le test assert vérifie le format BA26, pas l'acceptation par la plateforme, confirmée séparément par l'utilisateur.

QUESTIONS PROBABLES

A-t-on cassé SHA-256 ? Non. La fonction est utilisée dans le sens normal pour reconstruire un flux dont les entrées sont connues. Le script extrait ctf_unlock mais ne l'exécute pas.

SOURCES

TAPO · SOLVE.PY · POWERPOINT SLIDES 9–11

LIMITES ET PROVENANCE

La recette est vérifiée par lecture du script. Aucun déchiffrement ni extraction n'a été relancé pour ces slides.

Tapo — le résultat

À DIRE

« Le fichier flag.txt fourni contient ce texte, exactement identique à celui du PowerPoint. On dispose donc du résultat et d'un script qui décrit une procédure pour le retrouver depuis le dump. La soumission a été confirmée par la plateforme : le challenge est validé. La caméra avait gardé son secret et son mode d'emploi au même endroit. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l'idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Les fichiers reçus sont Resolution_CTF_Tapo.pptx, solve.py, ctf_unlock, flag.enc et flag.txt. flag.enc et flag.txt font chacun 35 octets. Le flag lu correspond au support. Le script vérifie la forme BA26{…}, mais cette vérification n'est pas une validation serveur.

QUESTIONS PROBABLES

Avons-nous reproduit le solve ici ? Non : les fichiers ont été lus, et le script inspecté. Le dump n'a pas été fourni dans cette demande, et aucune exécution n'a été réalisée. La reproduction annoncée est celle du PowerPoint source.

SOURCES

TAPO · FLAG.TXT FOURNI · SOUMISSION CONFIRMÉE PAR L'UTILISATEUR

LIMITES ET PROVENANCE

La concordance entre flag.txt et le PowerPoint est vérifiée. L'utilisateur a confirmé la soumission à la plateforme après la réception du PowerPoint ; cette confirmation remplace le statut provisoire du support. La provenance de la résolution et son auteur ne sont pas précisés ; aucune attribution à un agent n'est inventée.

Conclusion

À DIRE

« Les attaques sont différentes, mais on peut les raconter avec les mêmes questions. Quel était le verrou ? Quelle hypothèse était fragile ? Et quelle preuve nous permet de dire que cela a fonctionné ? C’est ce passage du résultat au raisonnement qui nous permet de nous approprier le travail des agents. »

DÉROULÉ DES ANIMATIONS

1. Présenter le défi.
2. → Faire apparaître l’idée suivante.
3. → Conclure avec la preuve ou la leçon.

POUR COMPRENDRE

Les schémas de cette présentation simplifient les mécanismes. Les notes rattachent chaque affirmation aux journaux et aux artefacts disponibles. Les captures annotées produites après l’événement sont identifiées comme telles.

QUESTIONS PROBABLES

Les chapitres peuvent être présentés séparément. Pour un créneau court, commencer par les embeddings, id_please et wrong_path. Les mécanismes les plus spécialisés restent détaillés dans leurs notes.

SOURCES

JOURNAUX ET ARTEFACTS CONSERVÉS · PRÉSENTATION DOCUMENTAIRE

LIMITES ET PROVENANCE

Schémas pédagogiques fondés sur les fichiers. Les animations ne représentent pas les temps réels du solve.